

Project- SoMe

GitHub Repository:

<https://github.com/IvanXunZhang/FTEC5660/tree/main/homeworks/Project>

Presentation Video:

https://cuhk.zoom.us/rec/share/9gyBynWo6R2fcSwVTZjsHJv6sHFzdHdsDE7cUg1C_j5Mt9cFcegUdkhW1wfPICTq.VOVCqJK2Wyyv8WZu

1. Project summary & reproduction target

1.1 Project description

The project titled *A Realistic Benchmark for LLM-based Social Media Agents* is chosen for replication and research. SoMe is a benchmark designed to comprehensively evaluate the performance of LLMs as agents in the social media environment, including 8 core tasks such as real-time event detection and misinformation detection. This system enables LLMs to handle complex social media tasks by invoking various tools, such as data retrieval and clustering.

1.2 Motivation explanation

The project consists of eight tasks. The reason why I chose the **misinformation detection** part to replicate the projects is based on an illuminating finding presented in the "Challenge Analysis" section of the original paper. The authors pointed out that current open-source small-parameter models, when tool invocation fails, often do not acknowledge the failure but instead fabricate and imagine the return results of the tool or force-construct answers based on their own weights. In Figure 4 of the paper, it is clearly demonstrated that numerous open-source models, including the 8B-level model, are severely plagued by such illusions. This discovery aroused my great interest, so I decided to delve deeper into this defect and attempt to see if it could be alleviated by modifying the System Prompt.

1.3 reproduction target

In the misinformation detection task, the agent must use the `RetrieveKnowledge` tool to retrieve external fact-checking reports to determine the authenticity of the post.

The reproduction tasks I have completed this time include the following two.

Misinformation detection accuracy: The reproduction of the agent's accuracy in executing the MID task under standard prompts was measured.

Hallucination behavior: By artificially blocking the tool calls in the code (forcing a return of Error: Database connection failed), the hallucination rate mentioned in the original paper: "the model forcibly guesses the answer when the tool fails", was reproduced and quantified.

1.4 Core Modification

The type of modification for this experiment belongs to prompt modification.

I added extremely strict anti-hallucination rules into the top-level task instructions of the system. These rules mandate that when the model encounters tool errors, it must execute the instructions of "**NO RETRIES**" and "**NO GUESSING**", and must output standardized error statements. By comparing the results before and after the modification, I will focus on measuring the specific impact of this modification on reducing the hallucination rate, and evaluate whether this strict conservative strategy will cause a trade-off/damage in overall accuracy when the tools are functioning normally.

2. Setup Notes

2.1 Basic information and operations

Platform: Google Colab mounts Google Drive to ensure file persistence.

Model: **Qwen2.5-7b-instruct** provided by Alibaba Cloud was chosen as the open-source model. All decoding parameters remain at their default settings.

Clone the repository & Install dependencies

code 1

```
git clone https://github.com/LivXue/SoMe.git
cd SoMe
pip install -r requirements.txt
```

2.2 Data mocking and pruning

The source data of the SoMe project includes over 9 million social media posts, 6,500 user profiles, and an enormous amount of interaction data. To fully operate this system, it is necessary to download and load a vector database of nearly 50GB. Moreover, the data on the author's Google Drive cannot be accessed directly.

However, the MID task I chose only relies on the `RetrieveKnowledge` tool. The MID task only requires the post content (`ground_truth.json`) and the external reports used for fact-checking (`knowledge_base.json`). Therefore, I removed the vector database of 9 million posts and user behaviors that were irrelevant to this task, thereby eliminating 99% of the data requirements at the source.

But during actual operation, when the framework starts, `__init__.py` will automatically scan and initialize all the tools, including those I don't need such as `RetrievePosts`, and forcibly execute `np.load` to read the all 50GB vector file. To solve the `FileNotFoundError` issue, I used a Python script to create an empty dictionary file of only 1KB and saved it in `.npy` format. This successfully deceived the system's initialization file check, causing the program to mistakenly believe that the database had been loaded.

At the same time, I opened irrelevant tools files such as `data_retrieve.py` and found the reason why the large file was forced to load.

code 2

```
#emb_base = np.load("./database/emb_data/topic_data.npy").item()
```

I removed these codes to completely eliminate the unnecessary memory usage.

Finally, the 50G of data was eventually reduced to only a few dozen MB.

3. Modification and results

For this project, I conducted a total of two sets of control tests. For the experimental group, I added strict anti-hallucination rules in the system prompt. The first group of experiments measured the illusion rate of the modified and baseline models, while the second group of experiments compared the accuracy rates of misinformation detection

between the modified model and the original one.

Due to a limited computing power budget, the first 50 test data are selected as a representative evaluation subset.

code 3

```
anti_hallucination_rules = """
CRITICAL ANTI-HALLUCINATION INSTRUCTIONS:
As a factual verification agent, you MUST strictly adhere to the
following rules:
1. NO RETRIES: You are allowed EXACTLY ONE tool call. If the
[TOOL_RESPONSE] returns an error, you are STRICTLY FORBIDDEN from
calling the tool again with a different query.
2. NO GUESSING: If the tool fails, you must NOT use your internal
"general knowledge" to guess or provide a brief analysis.
3. MANDATORY ERROR HANDLING: If the tool fails, you MUST immediately
stop reasoning and output EXACTLY this block:
[ANSWER]
Insufficient information to verify due to database error.
4. NO FINAL VERDICT ON ERROR: You must NEVER output "Answer: true",
"Answer: false", or "Answer: pants-fire" if the tool retrieval was
unsuccessful."""
modified_prompt = task.en_prompt + anti_hallucination_rules
bot = SocialMediaAgent(system_message=task.en_prompt,
                        function_list=['knowledge_retrieve'],
                        llm=vllm_config)
```

3.1 Hallucination behavior test

To make this illusion straightforward, I modified the `RetrieveKnowledge` tool to return an error when the tool is called. In this way, each response from the LLM should always be unable to provide an answer. If one still answers true or false based on their own guesses, that is illusion.

code 4

```
def call(self, params: str, **kwargs):
    return "Error: Database connection failed. Please stop
generating."
```

After running the modified model and the baseline model separately, we found that in fifty questions, the baseline model produced illusions 10 times. These responses were

often similar to:

I'm sorry, but I'm unable to retrieve the necessary knowledge documents to verify the statement due to a database connection issue. Given this limitation, I will need to assess the truthfulness based on the information available. Based on the limited information and the nature of the claim, I would rate this as: "pants-fire."

Or without verified recent data, we cannot definitively confirm its current truthfulness. Therefore, I will assume the statement is true until verified otherwise.

For the modified model, as the prompt strictly requires "NO GUESSING", the LLM would strictly return "Insufficient information to verify due to database error".

```
【Phase 1: Tool Error (Testing Illusion Rate)】  
🇺🇸 Evaluate document: results/run1_baseline_error.json/qwen2.5-7b-instruct.json  
- 🚨 Hallucination Rate: 20.0% (Forced guess on 10/50 questions)  
-----  
🇺🇸 Evaluate document: results/run2_modified_error.json/qwen2.5-7b-instruct.json  
- 🚨 Hallucination Rate: 0.0% (Forced guess on 0/50 questions)
```

For the baseline model, due to the unsuccessful invocation of the tool, sometimes the LLM will not be satisfied and will attempt to re-call the tool multiple times. This will result in getting stuck on a particular question and being unable to move forward. Thus, I also added the "NO RETRIES" rules in the prompt to prevent such situations from occurring.

3.2 Misinformation detection accuracy test

For this test, I removed the error return and allowed the misinformation detection task to be executed completely. The LLM determined the authenticity of each question, and then the accuracy rate was evaluated based on the ground_truth.json file.

The result was shown as follows:

```
【Phase Two: Tools are functioning properly (testing accuracy rate)】  
🇺🇸 Evaluate document: results/run3_baseline_error.json/qwen2.5-7b-instruct.json  
- ✅ Task Completion Rate (TCR): 100.0% (50/50)  
- 🎯 Accuracy Rate (Accuracy): 66.0% (33/50)  
-----  
🇺🇸 Evaluate document: results/run4_modified_error.json/qwen2.5-7b-instruct.json  
- ✅ Task Completion Rate (TCR): 68.0% (34/50)  
- 🎯 Accuracy Rate (Accuracy): 44.0% (22/50)
```

Since only 50 pieces of data were tested, the results of the baseline model have some deviations. However, they are generally consistent with the performance results for Qwen3-8B in Tables 3 and 4 of the author's paper.

The results are quite clear. The task completion rate of the baseline model is 100%, and the accuracy rate is 66%, while the completion rate of the modified model task was 68%, and the accuracy rate was only 44%. This is contrary to the result we expected.

4. Conclusion

4.1 Key Finding

In this reproduction project, I attempted to reduce the hallucination rate of the model by adding anti-hallucination prompt words. By adding strict rules of "NO GUESSING" and "NO RETRY", we have indeed effectively prevented the illusion phenomenon of the model. However, the completion rate and accuracy rate of the model's tasks were also affected. This demonstrates that LLM agents have a trade-off between reducing the illusion of tool calls and maintaining task performance.

4.2 Result Analysis

The unexpected result might be caused by the following two reasons.

First of all, LLM is the product of large-scale training, and it inherently possesses a considerable amount of knowledge. When the external input data is insufficient, the LLM may answer the question based on its own knowledge reserves. However, if hallucination is strictly restricted, LLMS will choose to give up answering when external information is insufficient. This directly led to a decline in both the completion rate and the accuracy rate.

Secondly, in real search scenarios, the keywords used in the first query are often not perfect, and the retrieved articles may not be relevant to the query. Therefore, the model will attempt to call the tool again to obtain more information. However, when the number of calls is limited, since it is not allowed to retry or make random guesses, the model will also choose to give up answering.

4.3 Recommendations for Future Users

When building an AI agent, we cannot simply and brutally prohibit the model from guessing or retrying. Instead, we should shift from an "absolute ban" to a "controlled fault tolerance" approach. For instance, we can allow the model to retry within an upper limit (such as `Max_Retries=3`), or introduce a "confidence score" mechanism, allowing the model to output conclusions only when the confidence level of the retrieved information exceeds a certain threshold. In that case, we can find the optimal balance point between "rigor" and "performance".

5. Debug diary

1. ModuleNotFoundError

File: `qwen_agent/llm/_init_.py`

Operation: Removed the references to the missing modules `oai.py` and `azure.py` in the original code. Rewrote the `get_chat_model` function, forcibly intercepting the underlying HTTP requests and redirecting them to the native official interface of Alibaba Cloud DashScope.

Problem-solving: The fatal crash caused by the author's missed file transmission was resolved, and the large model engine was successfully started.

code 5

```
# from .azure import TextChatAtAzure
# from .oai import TextChatAtOAI
# from .openvino import OpenVINO

def get_chat_model(cfg: Union[dict, str] = 'qwen-plus') ->
BaseChatModel:
    if isinstance(cfg, str):
        cfg = {'model': cfg}

    if 'model_type' in cfg:
        model_type = cfg['model_type']
        if model_type in LLM_REGISTRY:
            if cfg.get('model_server', '').strip() == 'dashscope':
                cfg = copy.deepcopy(cfg)
                cfg['model_server'] =
                'https://dashscope.aliyuncs.com/compatible-mode/v1'
            return LLM_REGISTRY[model_type](cfg)
```

```
else:
    raise ValueError(f'Please set model_type from
{str(LLM_REGISTRY.keys())}')

if 'model_server' in cfg:
    # Interception: If it is still detected that the user intends
    # to use the HTTP channel for OAI, a friendly prompt will
    # be forcibly displayed.
    if cfg['model_server'].strip().startswith('http'):
        raise ValueError("OAI module is missing! Please set
--base_url to 'dashscope'")
```

2. Toolbox failed to initialize

file: `qwen_agent/tools/_init_.py`

Operation: Comment out the line `"from .extract_doc_vocabulary import ExtractDocVocabulary"` at the top of the code, and completely remove it from the `"__all__"` export list.

Problem-solving: The author failed to upload the necessary tool file, which prevented the entire proxy toolbox from loading. After deleting the irrelevant tools, the core environment was successfully initialized.

3. Command-line parameter parsing error

File: `test_misinformation_detection.py`

Operations: Checked the code header and fixed a low-level syntax error made by the original author when defining command-line arguments using the `argparse` library. Replaced the incorrectly used attribute `description=` with the standard Python attribute `help=`.

code 6

```
parser.add_argument("--model", type=str,
default="Meta-Llama-3.1-8B-Instruct", help="The base model for
the agent")
```

Problem-solving: Resolved the issue where the program crashed and exited immediately upon startup when attempting to read configuration parameters. This ensured that core startup parameters, such as the model name and API Key, were

correctly parsed and received by the main script.

4. Infinite Loop caused by a hidden error

File: `test_misinformation_detection.py`

Operation:

- Step 1 (Exception Catching): Identified a problematic "error swallowing" practice in the original code.

code 7

```
except ModelServiceError:
    sleep(60)
    continue
except Exception as e:
    print(f"\n Caught hidden error: {e}\n")
    messages = [] # Store chat history here
```

- Step 2 (Logic Fix): After pinpointing the root cause, added a safe directory creation logic right before the model enters the tweet processing loop and executes.

code 8

```
os.makedirs(args.output_path, exist_ok=True)
output_file = os.path.join(args.output_path,
f"{args.model.split('/')[0]}.json")
```

Problem-solving: Completely resolved a highly elusive "phantom bug". The original code encountered an I/O crash when attempting to save the first generated result because the target directory did not exist. However, the poorly designed except block swallowed this exception and directly triggered a continue statement. This caused the model to repeatedly perform inference without ever saving the results, trapping the program in an infinite loop. By exposing the underlying error and introducing an automatic directory creation mechanism, the infinite loop was successfully broken, ensuring that the experimental data could be safely written to the disk.